

مذكرة شاملة في MySQL و SQL قواعد البيانات و

Database Fundamentals with SQL & MySQL

تغطي هذه المذكرة

Joins العلاقات والـ | CRUD عمليات | SQL مقدمة قواعد البيانات | أساسيات
العملي | مشاريع تطبيقية كاملة MySQL | الاستعلامات المتقدمة

الفصل الأول: مقدمة في قواعد البيانات

ما هي قواعد البيانات ولماذا نحتاجها؟ 1.1

تخيل معي متجر إلكتروني فيه مليون منتج وخمسة ملايين عميل وعشرات الآلاف من الطلبات يومياً. كيف تخزن هذه البيانات وتسترجعها في ثوانٍ؟ هنا تأتي قاعدة البيانات.

هي نظام منظم لتخزين واسترجاع البيانات بكفاءة عالية. لكن المهم مش التعريف — المهم إنك تفهم المشكلة اللي (Database) قاعدة البيانات بتحلها.

Files و Folders مشاكل التخزين بدون قاعدة بيانات — ليه مش

هتواجه مشاكل حقيقية فوراً، Text files أو Excel لو قررت تخزن بيانات عملاء متجرك في ملفات

- ياخذ دقائق. قاعدة البيانات تعمل نفس الشيء في ميلي ثانية Excel السرعة: البحث عن عميل واحد في مليون سطر في
- التكرار: نفس بيانات العميل ممكن تتكتب في أماكن كثيرة وتتعارض — في الفاتورة كتبت اسمه 'أحمد محمد' وفي الشحن 'أحمد م
- الأمان: أي حد يعرف يفتح الملف يقدر يعدل أي حاجة بدون تتبّع أو صلاحيات
- التزامن: لو اتنين بيعدلو نفس الملف في نفس الوقت، مين اللي هيتحفظ؟
- العلاقات: إزاي تربط الطلب بالعميل بالمنتج بالشحن في ملفات منفصلة؟

1.2 File System vs Database System

File System	Database System
بحث خطي — بيقرا كل سطر	فهرسة — بيوصل للبيانات مباشرة
لا يدعم الاستعلامات المعقدة	استعلامات قوية جاهزة — SQL
لا تزامن متعدد المستخدمين	Transactions — تزامن آمن
لا ضمانات سلامة البيانات	قواعد صارمة — Constraints
لا علاقات بين الملفات	Foreign Keys — علاقات منظمة
يدوي وصعب Backup	تلقائي ومنظم Backup

أنواع قواعد البيانات — عارف الفرق؟ 1.3

مش كل المشاكل بتتحل بنفس الأداة. فيه أنواع مختلفة من قواعد البيانات كل نوع مصمم لحالة استخدام مختلفة

متى تستخدمه	المثال	النوع
بيانات منظمة ذات علاقات — متاجر ERP بنوك، أنظمة	MySQL, PostgreSQL, SQLite	Relational (علائقية)
بيانات غير منظمة أو كميات ضخمة جداً — سوشيال ميديا	MongoDB, Cassandra, Redis	NoSQL
بيانات ذات علاقات معقدة — شبكات اجتماعية، توصيات	Neo4j, Amazon Neptune	Graph
بيانات مرتبطة بالوقت — مقاييس، أسعار IoT، أسهم	InfluxDB, TimescaleDB	Time Series
نماذج — AI للـ Embeddings تخزين اللغة، البحث الدلالي	Pinecone, Weaviate, Chroma	Vector Database

الصورة الكاملة — AI قواعد البيانات في الـ 1.4

يحتاج بيانات للتدريب، وهذه البيانات مخزنة في قاعدة بيانات AI وقواعد البيانات علاقة تكاملية عميقة. كل نموذج AI العلاقة بين

دورة حياة البيانات في أنظمة الذكاء الاصطناعي

← SQL تُستعمل وتُنظف باستخدام (SQL أو NoSQL) البيانات الخام تُجمع من مصادر متعددة — تُخزن في قاعدة بيانات SQL. نتائج الموديل تُخزن مجدداً في قاعدة البيانات ← AI تُستخدم لتدريب موديلات ← Python في DataFrames تُحوّل لـ! هو الجسر الرئيسي في هذه الدورة

RAG (Retrieval Augmented Generation) مثل Chatbot الحديث: عند بناء AI خاصة أصبحت جزءاً أساسياً من الـ Vector Databases ثم عند الاستعلام بتجيب أقرب الـ (Vector DB) متجهات رقمية (وبتخزن في Embeddings بياناتك بتتحول لـ Embeddings للسؤال.

وأشهر أنظمتها؟ DBMS ما هو 1.5

هو البرنامج الوسيط بينك وبين البيانات. هو اللي يبتحكم في التخزين والأمان (DBMS (Database Management System الـ وهو بيترجمها لعمليات على الملفات SQL والاستعلامات. أنت بتكتب

مميزاته وحالة استخدامه	DBMS نظام
Web Development مفتوح المصدر، سريع، أكثر انتشاراً في اختيارنا —	MySQL
أنواع بيانات متقدمة JSON أقوى وأكثر مرونة، يدعم	PostgreSQL
Android قاعدة بيانات في ملف واحد — مثالي للتطبيقات الصغيرة و	SQLite
مرن، مناسب للبيانات غير المنظمة، NoSQL	MongoDB
قوي جداً للشركات الكبيرة، مدفوع	Oracle

تحديداً؟ لماذا MySQL

مجاني ومفتوح المصدر. موارد تعليمية (YouTube, بدأت بيه Facebook, WordPress) الأكثر استخداماً في الويب MySQL بشكل عام SQL فيه متوافقة 90% مع باقي قواعد البيانات. تعلمه = تعلم SQL ضخمة. سريع في القراءة. أوامر

Table, Row, Column, Schema — المفاهيم الأساسية 1.6

هو سطر بيانات لكيان (Row) كل صف، Sheet هو (Table) لكن أقوى بكثير. كل جدول، Excel Workbook فكر في قاعدة البيانات كـ. هو خاصية محددة (Column) واحد، وكل عمود

مثال	التعريف	المفهوم
قاعدة بيانات متجر إلكتروني	مجموعة جداول مترابطة	Database
جدول العملاء، جدول المنتجات	بيانات كيان واحد منظمة في صفوف وأعمدة	Table
بيانات عميل واحد بالكامل	بيانات عنصر واحد	Row (Record)
اسم العميل، بريده الإلكتروني	خاصية معينة لكل العناصر	Column (Field)
تصميم كل الجداول وعلاقاتها	هيكل قاعدة البيانات وتعريف الجداول	Schema
customer_id = 1, 2, 3, ...	معرف فريد لكل صف	Primary Key

SQL الفصل الثاني: أساسيات

ولماذا هي اللغة الأساسية للبيانات؟ SQL ما هي 2.1

هي — Python هي لغة التواصل مع قواعد البيانات العلائقية. مش لغة برمجة كلاسيكية زي (SQL (Structured Query Language يقرر هو DBMS يعني بتقول 'ماذا تريد' مش 'كيف تحصل عليه'. 'أنت بتكتب' أعطني كل العملاء من مصر 'والـ (Declarative) لغة تصريحية. إزاي يجيبهم بأكفا طريقة

المنظومة الكاملة — SQL أنواع أوامر

الأوامر والغرض	النوع
CREATE, DROP, ALTER, RENAME, TRUNCATE — بناء وتعديل هيكل الجداول	تعريف الهيكل — DDL
INSERT, UPDATE, DELETE — إضافة وتعديل وحذف البيانات	تعديل البيانات — DML
SELECT — قراءة واسترجاع البيانات	الاستعلام — DQL
GRANT, REVOKE — من يقدر يعمل إيه في قاعدة البيانات	التحكم والصلاحيات — DCL
COMMIT, ROLLBACK, SAVEPOINT — ضمان اتساق البيانات	إدارة المعاملات — TCL

لماذا تهتمنا؟ — (Data Types) أنواع البيانات 2.2

يأخذ بقدر ما VARCHAR(255)، bytes يأخذ 4 INT. اختيار نوع البيانات الصح مش بس شكل — هو يؤثر على الأداء والتخزين والسلامة. إستفقد الأصفار البادئة INT تحتاج. لو خزنت رقم هاتف كـ

مثال الاستخدام	الوصف	نوع البيانات
العمر، الكميات، المعرفات	(M إلى 2 -2M) أعداد صحيحة	INT
معرفات في أنظمة ضخمة	أعداد صحيحة كبيرة جداً	BIGINT
إحداثيات، قياسات علمية	أعداد عشرية بدقة متغيرة	FLOAT / DOUBLE
الأسعار والمبالغ المالية — لا تستخدم للمال FLOAT	أعداد عشرية بدقة ثابتة	DECIMAL(p,s)
(M/F) الجنس، (EG, US) رموز الدول	حرف n نص ثابت الطول دائماً	CHAR(n)
الاسم، البريد الإلكتروني	حرف n نص متغير الطول حتى	VARCHAR(n)
التعليقات، الأوصاف الطويلة	نص طويل جداً بدون حد محدد	TEXT

DATE	YYYY-MM-DD تاريخ فقط	تاريخ الميلاد
DATETIME	YYYY-MM-DD HH:MM:SS تاريخ ووقت	وقت إنشاء الطلب
TIMESTAMP	UTC لكن بتوقيت DATETIME مثل	تتبع التعديلات تلقائياً
BOOLEAN	MySQL 1 أو 0 (في) صح أو غلط	هل الحساب مفعل؟
JSON	مباشرة في العمود JSON بيانات	إعدادات مرنة، بيانات متداخلة
BLOB	(Binary) بيانات ثنائية	صور، ملفات — لكن يُفضل تخزين المسار فقط

مش صفر ومش فراغ NULL — تحذير مهم

يحتاج معاملة خاصة في الاستعلامات: لا NULL. يعني 'لا توجد قيمة معروفة'. 0 هو صفر حقيقي ". هو نص فارغ حقيقي NULL. هذا مصدر أخطاء شائعة جداً. NULL = NULL أي عملية حسابية مع NULL IS NULL بل NULL = تستخدم

إنشاء وإدارة قواعد البيانات والجداول 2.3

إنشاء قاعدة بيانات وجدول

نبدأ بمثال عملي حقيقي — سننشئ قاعدة بيانات لنظام متجر إلكتروني

```
-- إنشاء قاعدة البيانات
CREATE DATABASE ecommerce_db
CHARACTER SET utf8mb4 -- Emoji يدعم العربية والـ
COLLATE utf8mb4_unicode_ci; -- مقارنة الحروف بشكل صحيح

-- تحديد قاعدة البيانات للعمل بها
USE ecommerce_db;

-- إنشاء جدول العملاء
CREATE TABLE customers (
  customer_id INT AUTO_INCREMENT PRIMARY KEY, -- معرف فريد تلقائي
  full_name VARCHAR(100) NOT NULL, -- الاسم مطلوب
  email VARCHAR(150) UNIQUE NOT NULL, -- لا تكرار في الإيميل
  phone VARCHAR(20) NULL, -- الهاتف اختياري
  birth_date DATE NULL,
  gender CHAR(1) CHECK (gender IN ('M','F')), -- M أو F فقط
  balance DECIMAL(10,2) DEFAULT 0.00, -- رصيد افتراضي صفر
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- وقت إنشاء تلقائي
);
```

القيدود التي تحمي بياناتك — 2.4 Constraints

Constraints تلفائياً على البيانات. بدونها، قاعدة بياناتك مجرد تخزين بدون ضمانات. مع الـ DBMS هي قواعد يفرضها الـ Constraints الـ قاعدة بياناتك تحمي نفسها من البيانات الغلط.

الغرض والمثال	القيد (Constraint)
— NULL معرّف فريد لكل صف — لا يتكرر ولا يكون customer_id	PRIMARY KEY
يشير لـ customer_id يحتوي order — يربط جدولين customers	FOREIGN KEY
البريد الإلكتروني — NULL قيمة فريدة لكن ممكن تكون	UNIQUE
العمود لازم يحتوي قيمة — الاسم مثلاً	NOT NULL
is_active = TRUE — قيمة افتراضية لو لم يُحدد شيء	DEFAULT
gender IN ('M','F') — شرط يجب أن يتحقق	CHECK
ID قيمة تزداد تلفائياً — معرفات	AUTO_INCREMENT

CRUD الفصل الثالث : عمليات

CRUD — الصورة الكاملة

هذه الأربع عمليات هي كل ما يحتاجه أي تطبيق للتعامل مع بياناته. موقع. CRUD = Create, Read, Update, Delete. Facebook: Create = جديد post نشر، Read = عرض الـ feed، Update = تعديل التعليق، Delete = كل شيء يرجع لـ CRUD.

إضافة البيانات — 3.1 INSERT

هي الطريقة الوحيدة لإضافة بيانات جديدة. المنطق: حدد الجدول، حدد الأعمدة، أعط القيم بنفس الترتيب INSERT INTO

```
-- إضافة عميل واحد
INSERT INTO customers (full_name, email, phone, gender, birth_date)
VALUES ('أحمد محمد علي', 'ahmed@email.com', '01012345678', 'M', '1990-05-15');

-- منفصلة INSERT أسرع بكثير من إضافة أكثر من عميل دفعة واحدة
INSERT INTO customers (full_name, email, gender) VALUES
('سارة أحمد', 'sara@email.com', 'F'),
('محمود خالد', 'mahmoud@email.com', 'M'),
('نورا حسن', 'noura@email.com', 'F');
```

قراءة البيانات — الأقوى على الإطلاق — 3.2 SELECT

SELECT من أبسط استعلام لأكثر استعلام تعقيداً — كله يبدأ بـ SQL. هو أكثر أمر ستستخدمه في حياتك مع SELECT

```
(!على جداول كبيرة Production تجنب في الـ) قراءة كل شيء --
SELECT * FROM customers;

-- قراءة أعمدة محددة فقط (أسرع وأوضح)
SELECT full_name, email, created_at FROM customers;

-- إزالة التكرار - DISTINCT
فقط بدون تكرار F و M يرجع --
SELECT DISTINCT gender FROM customers;

-- التصفية - WHERE
SELECT * FROM customers
WHERE gender = 'F' AND is_active = TRUE;

-- نطاق قيم - BETWEEN
SELECT * FROM customers
WHERE birth_date BETWEEN '1990-01-01' AND '2000-12-31';

-- قائمة قيم - IN
```



```

SELECT * FROM customers
WHERE customer_id IN (1, 5, 10, 15);

-- LIKE - البحث في النصوص
SELECT * FROM customers WHERE full_name LIKE 'يبدأ بأحمد'; أحمد
SELECT * FROM customers WHERE email LIKE '%@gmail.com'; -- gmail ينتهي بـ
SELECT * FROM customers WHERE full_name LIKE '%يحتوي محمد'; محمد

-- IS NULL - قيم فارغة
SELECT * FROM customers WHERE phone IS NULL;
SELECT * FROM customers WHERE phone IS NOT NULL;

-- ORDER BY - الترتيب
SELECT * FROM customers ORDER BY full_name ASC; -- أبجدياً تصاعدي
SELECT * FROM customers ORDER BY created_at DESC; -- الأحدث أولاً

-- LIMIT - تحديد عدد النتائج
SELECT * FROM customers ORDER BY created_at DESC LIMIT 10; -- آخر 10 عملاء
SELECT * FROM customers LIMIT 10 OFFSET 20; -- (الصفحة الثالثة) 10 عناصر لكل صفحة
SELECT * FROM customers LIMIT 10, 20; -- (الصفحة الثالثة) 10 عناصر لكل صفحة

```

تعديل وحذف البيانات — UPDATE و DELETE 3.3

إستعمل أو تحذف كل الجدول، WHERE بدون UPDATE و DELETE في WHERE قاعدة ذهبية: دائماً اكتب

```

-- تحديث بيانات عميل محدد
UPDATE customers
SET phone = '01198765432', balance = 100.00
WHERE customer_id = 1; -- WHERE لا تنس الـ

-- تحديث بناءً على شرط
UPDATE customers
SET is_active = FALSE
WHERE last_login < '2024-01-01'; -- إلغاء تفعيل الحسابات القديمة

-- حذف سجل محدد
DELETE FROM customers WHERE customer_id = 5;

-- TRUNCATE - حذف كل البيانات (أسرع من DELETE بدون WHERE)
TRUNCATE TABLE temp_logs; -- AUTO_INCREMENT يحذف كل البيانات ويعيد ضبط

```

DELETE vs TRUNCATE vs DROP: الفرق الحاسم

DELETE: يحذف صفوف محددة (مع WHERE) أو كلها (بدون WHERE)، يمكن Rollback. TRUNCATE: يحذف كل
يُحذف الجدول نفسه بالكامل مع هيكله. استخدم DROP. Rollback. صعب، AUTO_INCREMENT البيانات بسرعة، يعيد ضبط
لإزالة جدول نهائياً DROP، لتفريغ جدول مؤقت TRUNCATE، للحذف الانتقائي DELETE.

Normalization الفصل الرابع: العلاقات بين الجداول والـ

لماذا نحتاج العلاقات؟ — مشكلة التكرار 4.1

تخيل أنك بتخزن بيانات الطلبات في جدول واحد وكل طلب يحتوي اسم العميل وعنوانه وإيميله. لو العميل عنده 100 طلب، بياناته متكررة 100
Foreign Keys. مرة! لو غير إيميله، لازم تعدل 100 سطر. الحل: فصل البيانات في جداول مختلفة وربطها بـ

أنواع العلاقات — كيف تفكر فيها؟

نوع العلاقة	المثال والتطبيق
One to One (1:1)	ID شخص لديه جواز سفر واحد فقط — جدولان مرتبطان بنفس الـ
One to Many (1:N)	عميل واحد له أوامر كثيرة — الأكثر شيوعاً في كل الأنظمة
Many to Many (N:M)	الطلاب والمواد — طالب في مواد كثيرة، مادة فيها طلاب كثيرون

كود تطبيقي — ربط الجداول

```
-- (One to Many) جدول الطلبات مرتبط بجدول العملاء
CREATE TABLE orders (
  order_id      INT      AUTO_INCREMENT PRIMARY KEY,
  customer_id   INT      NOT NULL,
  total_amount  DECIMAL(10,2) NOT NULL,
  status        VARCHAR(20)  DEFAULT 'pending',
  order_date    TIMESTAMP  DEFAULT CURRENT_TIMESTAMP,

  -- ربط مع جدول العملاء
  FOREIGN KEY (customer_id)
  REFERENCES customers(customer_id)
  ON DELETE RESTRICT -- لا تحذف عميل له طلبات
  ON UPDATE CASCADE  -- لا تغير الـ ID لو تغير الـ
);

-- Many to Many (Junction Table)
CREATE TABLE order_items (
  order_id      INT NOT NULL,
  product_id    INT NOT NULL,
```

```

quantity    INT    NOT NULL,
price       DECIMAL(10,2),
PRIMARY KEY (order_id, product_id), -- مفتاح مركب
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

4.2 ON DELETE — ماذا يحدث حين حذف البيانات المرتبطة؟

الخيار	ما يحدث ومتى تستخدمه
RESTRICT (الافتراضي)	يمنع الحذف لو فيه بيانات مرتبطة — للبيانات المهمة (عملاء لديهم طلبات)
CASCADE	يحذف البيانات المرتبطة تلقائياً — للتعليقات عند حذف المنشور
SET NULL	للبينات الاختيارية — NULL يساوي Foreign Key يجعل الـ
NO ACTION	Transaction لكن الفحص يتم في نهاية الـ RESTRICT مثل

4.3 Normalization — تنظيم قاعدة البيانات علمياً

Normal هي عملية تنظيم جداول قاعدة البيانات لتقليل التكرار وتحسين الاتساق. مبنية على مجموعة من القواعد اسمها Normalization الـ Forms.

النماذج الطبيعية — كل مرحلة تبني على السابقة

النموذج	الشرط	المشكلة التي يحلها
1NF	كل خلية تحتوي قيمة واحدة فقط (لا قوائم)	— 'خلية تحتوي 'أحمر، أزرق، أخضر' تكسير لثلاث صفوف
2NF	كل — Partial Dependency لا يوجد عمود يعتمد على المفتاح الأساسي بالكامل	في جدول طلبات، اسم المنتج يعتمد على product_id فقط — انقله لجدول منفصل
3NF	كل — Transitive Dependency لا يوجد لا عمود يعتمد على عمود آخر غير المفتاح	zip_code → city → country — منفصل addresses أنشئ جدول
BCNF	كل — NF شكل أقوى من 3 Determinant هو Candidate Key	نادر الاحتياج إليه في التطبيقات العملية

متى تكسر القواعد؟ — Normalization vs Denormalization

أحياناً Data Warehouses، والـ AI ممتازة لضمان سلامة البيانات وتقليل التكرار. لكن في أنظمة Normalization بين 5 جداول، تخزين البيانات في جدول (JOIN) دمج الجداول عن عمد (أسرع للاستعلامات الضخمة. مثال: بدل Denormalization Denormalized schemas غالباً تستخدم Redshift و Snowflake مثل Analytical databases واحد كبير. الـ

الفصل الخامس: الاستعلامات المتقدمة

5.1 الدوال التجميعية (Aggregate Functions)

وهي مرتبطة مباشرة بما تعلمناه في — SQL الدوال التجميعية بتحسب قيمة واحدة من مجموعة صفوف. هي الأساس للتحليل والإحصاء داخل الإحصاء الوصفي.

```
-- COUNT — عد الصفوف
SELECT COUNT(*) AS total_customers FROM customers;
SELECT COUNT(phone) AS customers_with_phone FROM customers; -- NULL يتجاهل

-- SUM, AVG, MAX, MIN
SELECT
    SUM(total_amount) AS total_revenue,
    AVG(total_amount) AS average_order,
    MAX(total_amount) AS largest_order,
    MIN(total_amount) AS smallest_order
FROM orders
WHERE status = 'completed';
```

5.2 التجميع والتصفية — HAVING و GROUP BY

لكن WHERE مثل HAVING. على كل مجموعة Aggregate Function يجمع الصفوف المنتشابهة في مجموعة ويطبق GROUP BY بعده. HAVING، يفلتر قبل التجميع WHERE — بعد التجميع

```
-- مجموع مبيعات كل عميل
SELECT
    c.full_name,
    COUNT(o.order_id) AS order_count,
    SUM(o.total_amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.full_name -- لازم تذكر كل عمود مش في
HAVING SUM(o.total_amount) > 1000 -- فقط العملاء الذين أنفقوا أكثر من 1000
ORDER BY total_spent DESC;
```

5.3 Subqueries — استعلام داخل استعلام

داخل استعلام آخر. يبساعدك تحل مشاكل معقدة بخطوات منطقية. تخيل إنك تريد العملاء الذين أنفقوا أكثر SELECT هو استعلام Subquery من متوسط الإنفاق — لازم أولاً تحسب المتوسط ثم تقارن

```
عملاء أنفقوا أكثر من المتوسط العام --
```

```

SELECT full_name, balance
FROM customers
WHERE balance > (
SELECT AVG(balance) FROM customers -- Subquery يحسب المتوسط
);

-- Subquery في FROM (Derived Table)
SELECT dept, avg_salary
FROM (
SELECT department AS dept, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
) AS dept_stats
WHERE avg_salary > 5000;

```

SQL منطق شرطي داخل — 5.4 CASE WHEN

يبسمح لك تضيف منطق شرطي مباشرة في الاستعلام SQL. داخل IF/ELSE هو الـ CASE WHEN.

```

SELECT
full_name,
balance,
CASE
WHEN balance >= 10000 THEN 'VIP'
WHEN balance >= 5000 THEN 'Gold'
WHEN balance >= 1000 THEN 'Silver'
ELSE 'Regular'
END AS customer_tier
FROM customers;

```

SQL أقوى أداة في — 5.5 Window Functions

لكن بدون تجميع الصفوف في مجموعة (الـ Window) تحسب قيمة لكل صف بناءً على مجموعة صفوف مرتبطة به Window Functions الـ. هذا يعني إنك تقدر تعرض تفاصيل كل صف مع القيم المحسوبة. GROUP BY واحدة كـ.

```

-- ROW_NUMBER - الصفوف
SELECT
full_name,
total_spent,
ROW_NUMBER() OVER (ORDER BY total_spent DESC) AS rank
FROM customer_sales;

-- RANK - (الترتيب مع تكرار) لو تعادل، نفس الرقم
SELECT
product_name,
sales,

```

```

RANK() OVER (PARTITION BY category ORDER BY sales DESC) AS rank_in_category
FROM product_sales;

-- SUM تراكمي (Running Total)
SELECT
    order_date,
    total_amount,
    SUM(total_amount) OVER (ORDER BY order_date) AS running_total
FROM orders;

```

5.6 CTE — Common Table Expressions

يُسمح لك تعرّف استعلام مؤقت وتسميته، ثم استخدامه في الاستعلام الرئيسي كأنه جدول. هو بديل أنظف وأسهل (WITH clause) CTE الـ المتداخلة Subqueries قراءة من الـ

```

-- تبسيط استعلام معقد CTE
WITH active_customers AS (
    -- أولاً: العملاء النشطين
    SELECT customer_id, full_name, email
    FROM customers
    WHERE is_active = TRUE
),
customer_totals AS (
    -- ثانياً: إجمالي إنفاق كل عميل
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
)
-- أخيراً: ادماج النتيجة
SELECT ac.full_name, ct.total_spent
FROM active_customers ac
JOIN customer_totals ct ON ac.customer_id = ct.customer_id
ORDER BY ct.total_spent DESC;

```

بشكل احترافي Joins الفصل السادس :ال

Joins الصورة الكاملة — لماذا

هي الطريقة لاسترجاع البيانات من جداول متعددة في استعلام واحد Joins جعلت بياناتنا في جداول منفصلة. ال Normalization ال
لا معنى لها. هما وجهان لعملة واحدة Normalization Joins بدون

الفروق الحاسمة — Joins أنواع ال 6.1

```
-- (طلبات 8 orders و عملاء 5 customers: نفترض وجود جدولين --  
-- عملاء لديهم طلبات، 1 بدون طلبات، 2 طلبات بعميل حذف 4 --  
  
-- INNER JOIN — الجانبين في التطابقات فقط  
-- النتيجة: 6 صفوف ) الطلبات ذات العملاء الموجودين )  
SELECT c.full_name, o.order_id, o.total_amount  
FROM customers c  
INNER JOIN orders o ON c.customer_id = o.customer_id;  
  
-- LEFT JOIN — كل الجهة اليسرى +التطابقات من اليمين  
-- (لعميل بلا طلبات NULL، كل العملاء +طلباتهم) النتيجة: 9 صفوف --  
SELECT c.full_name, o.order_id  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id;  
  
-- RIGHT JOIN — عكس LEFT JOIN  
SELECT c.full_name, o.order_id  
FROM customers c  
RIGHT JOIN orders o ON c.customer_id = o.customer_id;  
  
-- SELF JOIN — الجدول يjoin مع نفسه  
-- مثال :موظفون ومديريهم في نفس الجدول --  
SELECT e.full_name AS employee, m.full_name AS manager  
FROM employees e  
JOIN employees m ON e.manager_id = m.employee_id;
```

ربط أكثر من جدولين — 6.2 Multi-Table Join

```
-- ربط 4 جداول :عملاء +طلبات +عناصر الطلبات +منتجات --  
SELECT  
c.full_name AS customer_name,  
o.order_id,  
p.product_name,  
oi.quantity,  
oi.quantity * oi.price AS line_total
```

```

FROM customers c
INNER JOIN orders o      ON c.customer_id = o.customer_id
  INNER JOIN order_items oi ON o.order_id    = oi.order_id
  INNER JOIN products p    ON oi.product_id = p.product_id
WHERE o.status = 'completed'
ORDER BY o.order_id;

```

متى تستخدمه	Join نوع الـ
تريد فقط البيانات المتطابقة في كلا الجدولين — الأكثر شيوعاً	INNER JOIN
تريد كل بيانات الجدول الأيسر حتى لو لا يوجد تطابق — تقرير عملاء بدون طلبات	LEFT JOIN
بتبديل ترتيب الجداول LEFT JOIN نادر — يمكن استبداله بـ	RIGHT JOIN
ضرب ديكارتي — كل صف مع كل صف — مفيد لتوليد بيانات تجريبية	CROSS JOIN
البيانات الهرمية — المدبرون والموظفون في نفس الجدول	SELF JOIN

عملياً MySQL الفصل السابع: التعامل مع

MySQL Python ربط 7.1

mysql-connector-python: مباشرةً. هناك مكتبتان رئيسيتان Python من MySQL نتعامل مع، Data Science والـ AI في مشاريع الـ SQLAlchemy و.

```
# تثبيت المكتبات
# pip install mysql-connector-python
# pip install sqlalchemy pandas

import mysql.connector
import pandas as pd

# الاتصال بقاعدة البيانات
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='your_password',
    database='ecommerce_db'
)

cursor = conn.cursor()

# DataFrame تنفيذ استعلام وتحويل النتيجة لـ
query = '''
SELECT c.full_name, COUNT(o.order_id) AS orders, SUM(o.total_amount) AS total
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id
ORDER BY total DESC
'''

df = pd.read_sql(query, conn)
print(df.head(10))

# SQL Injection لمنع Parameterized Queries استخدم إضافة بيانات بأمان
insert_query = 'INSERT INTO customers (full_name, email) VALUES (%s, %s)'
cursor.execute(insert_query, ('جديد تجريبي', 'test@email.com'))
conn.commit() -- مهم جداً لحفظ التغييرات

cursor.close()
conn.close()
```

الطريقة الاحترافية — SQLAlchemy 7.2

SQL بدون كتابة Python objects بتسمح لك تتعامل مع قاعدة البيانات كـ Python الأكثر استخداماً في ORM هي الـ SQLAlchemy مباشرة.

```
from sqlalchemy import create_engine, text
import pandas as pd

# إنشاء الاتصال
engine = create_engine('mysql+mysqlconnector://root:password@localhost/ecommerce_db')

# قراءة جدول كامل
df = pd.read_sql_table('customers', engine)

# استعلام مخصص
df = pd.read_sql_query('SELECT * FROM orders WHERE status = "completed"', engine)

# إلى قاعدة البيانات DataFrame كتابة
new_data = pd.DataFrame({
    'full_name': ['عميل جديد'],
    'email': ['new@email.com']
})

new_data.to_sql('customers', engine, if_exists='append', index=False)
```

7.3 CSV التعامل مع ملفات

أو تصديرها MySQL إلى CSV كثيراً ما ستحتاج لاستيراد بيانات من 'Data Science' في مشاريع الـ

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine('mysql+mysqlconnector://root:password@localhost/ml_db')

# استيراد CSV إلى MySQL
df = pd.read_csv('customers_data.csv')
df.to_sql('customers', engine, if_exists='replace', index=False)
print(f'تم استيراد {len(df)} سجل بنجاح')

# تصدير استعلام إلى CSV
df = pd.read_sql('SELECT * FROM orders WHERE YEAR(order_date) = 2024', engine)
df.to_csv('orders_2024.csv', index=False, encoding='utf-8-sig')
```

الفصل الثامن: المشاريع التطبيقية

مشروع 1 — نظام إدارة طلاب 8.1

سنبنى قاعدة بيانات كاملة لإدارة طلاب جامعة تتضمن طلاب، مواد، درجات، أساتذة

```
-- الهيكل الكامل
CREATE DATABASE university_db;
USE university_db;

CREATE TABLE students (
  student_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  major VARCHAR(50),
  gpa DECIMAL(3,2) DEFAULT 0.00,
  enrolled_at DATE DEFAULT (CURRENT_DATE)
);

CREATE TABLE courses (
  course_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  credits INT DEFAULT 3,
  professor VARCHAR(100)
);

-- Many to Many: طالب ينتسب لمواد كثيرة
CREATE TABLE enrollments (
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  grade DECIMAL(4,1) NULL,
  semester VARCHAR(20),
  PRIMARY KEY (student_id, course_id, semester),
  FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

-- لكل طالب GPA استعلام
SELECT
  s.name,
  COUNT(e.course_id) AS courses_taken,
  AVG(e.grade) AS calculated_gpa
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id
```

```
ORDER BY calculated_gpa DESC;
```

مشروع 2 — نظام متجر إلكتروني كامل 8.2

هذا المشروع يجمع كل ما تعلمناه — جداول، علاقات، استعلامات متقدمة

```
-- 1. منتجات وفئات
CREATE TABLE categories (
    cat_id    INT AUTO_INCREMENT PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    parent_id INT NULL, -- للفئات الفرعية (Self Reference)
    FOREIGN KEY (parent_id) REFERENCES categories(cat_id)
);

CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(200) NOT NULL,
    price       DECIMAL(10,2) NOT NULL,
    stock       INT DEFAULT 0,
    cat_id      INT,
    FOREIGN KEY (cat_id) REFERENCES categories(cat_id)
);

-- 2. الاستعلامات التحليلية
-- أكثر 5 منتجات مبيعاً
SELECT
    p.name AS product,
    SUM(oi.quantity) AS total_sold,
    SUM(oi.quantity * oi.price) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN orders o  ON oi.order_id = o.order_id
WHERE o.status = 'completed'
GROUP BY p.product_id
ORDER BY total_sold DESC
LIMIT 5;

-- تحليل مبيعات شهرية
SELECT
    YEAR(order_date) AS year,
    MONTH(order_date) AS month,
    COUNT(*) AS orders_count,
    SUM(total_amount) AS monthly_revenue
FROM orders
WHERE status = 'completed'
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year, month;
```

8.3 Python + MySQL + AI مشروع

حقيقي — تحليل بيانات المبيعات وبناء موديل تنبؤ AI الآن نربط قاعدة البيانات بمشروع

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

engine = create_engine('mysql+mysqlconnector://root:pass@localhost/ecommerce_db')

# 1. جلب البيانات من MySQL
query = '''
SELECT
    YEAR(o.order_date) AS year,
    MONTH(o.order_date) AS month,
    COUNT(o.order_id) AS orders_count,
    AVG(o.total_amount) AS avg_order_value,
    SUM(o.total_amount) AS monthly_revenue
FROM orders o
WHERE o.status = 'completed'
GROUP BY YEAR(o.order_date), MONTH(o.order_date)
ORDER BY year, month
'''
df = pd.read_sql(query, engine)

# 2. إعداد البيانات للـ AI
X = df[['year', 'month', 'orders_count', 'avg_order_value']]
y = df['monthly_revenue']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 3. بناء موديل التنبؤ
model = LinearRegression()
model.fit(X_train, y_train)

score = model.score(X_test, y_test)
print(f'دقة الموديل: {score:.2%}')

# 4. حفظ التوقعات في قاعدة البيانات
predictions = pd.DataFrame({
    'year': X_test['year'],
    'month': X_test['month'],
    'predicted_revenue': model.predict(X_test),
    'actual_revenue': y_test
```

```

    })
    predictions.to_sql('revenue_predictions', engine, if_exists='replace', index=False)
    print('!تم حفظ التوقعات في قاعدة البيانات')

```

ملخص الأوامر الأساسية

الأمـر / المفهوم	الغرض	متى تستخدمه
CREATE TABLE	إنشاء جدول جديد بهيكله الكامل	مرة واحدة عند تصميم قاعدة البيانات
INSERT INTO	إضافة صف جديد للجدول	كل مرة تضيف بيانات جديدة
SELECT ... FROM	قراءة واسترجاع البيانات	في كل استعلام — الأكثر استخداماً
WHERE	تصفية الصفوف بشرط	مع SELECT, UPDATE, DELETE
UPDATE ... SET	تعديل قيم في صفوف موجودة	عند تحديث بيانات
DELETE FROM	حذف صفوف محددة	WHERE عند الحذف الانتقائي — لا تنس
INNER JOIN	ربط جدولين — التطابقات فقط	الأكثر شيوعاً في ربط الجداول
LEFT JOIN	كل اليسار + التطابقات من اليمين	تقارير شاملة حتى بدون تطابق
GROUP BY	تجميع الصفوف لحساب إحصاءات	مع COUNT, SUM, AVG, MAX, MIN
HAVING	GROUP BY تصفية بعد	Aggregate عندما تريد شرط على نتيجة
ORDER BY	ترتيب النتائج	ASC أو DESC دائماً في النهاية مع
LIMIT	تحديد عدد الصفوف المرجعة	وأسرع نتائج Pagination
FOREIGN KEY	ربط جدولين وضمان الاتساق	عند تصميم العلاقات بين الجداول
INDEX	تسريع البحث في الأعمدة	على الأعمدة الكثيرة ما تُستخدم في WHERE
CTE (WITH)	تعريف استعلام مؤقت مسمى	لتبسيط استعلامات معقدة

AI الخلاصة النهائية — قواعد البيانات و

يُمر بـ AI ليست مجرد أداة لتخزين البيانات. هي الجسر الحقيقي بين البيانات الخام وموديلات الذكاء الاصطناعي. كل مشروع SQL تدريب الموديل ← حفظ ← Python/Pandas تحليلها بـ ← SQL استعلامها وتنظيفها بـ ← DB جمع البيانات ← تخزينها في حقيقي AI قدرة على التعامل مع البيانات في أي مشروع = SQL إتقان DB. النتائج في